Continuous exposures with propensity scores

Malcolm Barrett

Stanford University

Warning! Propensity score weights are sensitive to positivity violations for continuous exposures.

The story so far

Propensity score weighting

1. Fit a propensity model predicting exposure x, x + z where z is all covariates
2. Calculate weights
3. Fit an outcome model estimating the effect of x on y weighted by the propensity score

Continous exposures

1. Use a model like `lm(x ~ z)` for the propensity score model.
2. Use `wt_ate()` with `.fitted`; transforms using `dnorm()` to get on probability-like scale.
3. Apply the weights to the outcome model as normal!

Alternative: quantile binning

1. Bin the continuous exposure into quantiles and use categorical regression like a multinomial model to calculate probabilities.
2. Calculate the weights where the propensity score is the probability you fall into the quantile you actually fell into. Same as the binary ATE!
3. Same workflow for the outcome model

1. Fit a model for `exposure ~ confounders`

```
model <- lm(
  exposure ~ confounder_1 + confounder_2,
  data = df
)
```

2. Calculate the weights with `wt_ate()`

```
model |>
  augment(data = df) |>
  # wt_ate() estimates the SD from the model
  mutate(wts = wt_ate(
    .fitted,
    exposure,
    exposure_type = "continuous"
  ))
```

Does change in smoking intensity (`smkintensity82_71`) affect weight gain among lighter smokers?

```
nhefs_light_smokers <- nhefs_complete |>
  filter(smokeintensity <= 25)
```

1. Fit a model for `exposure ~ confounders`

```r
nhefs_model <- lm(
  smkintensity82_71 ~ sex + race + age + I(age^2) +
    education + smokeintensity + I(smokeintensity^2) +
    smokeyrs + I(smokeyrs^2) + exercise + active +
    wt71 + I(wt71^2),
  data = nhefs_light_smokers
)
```

2. Calculate the weights with `wt_ate()`

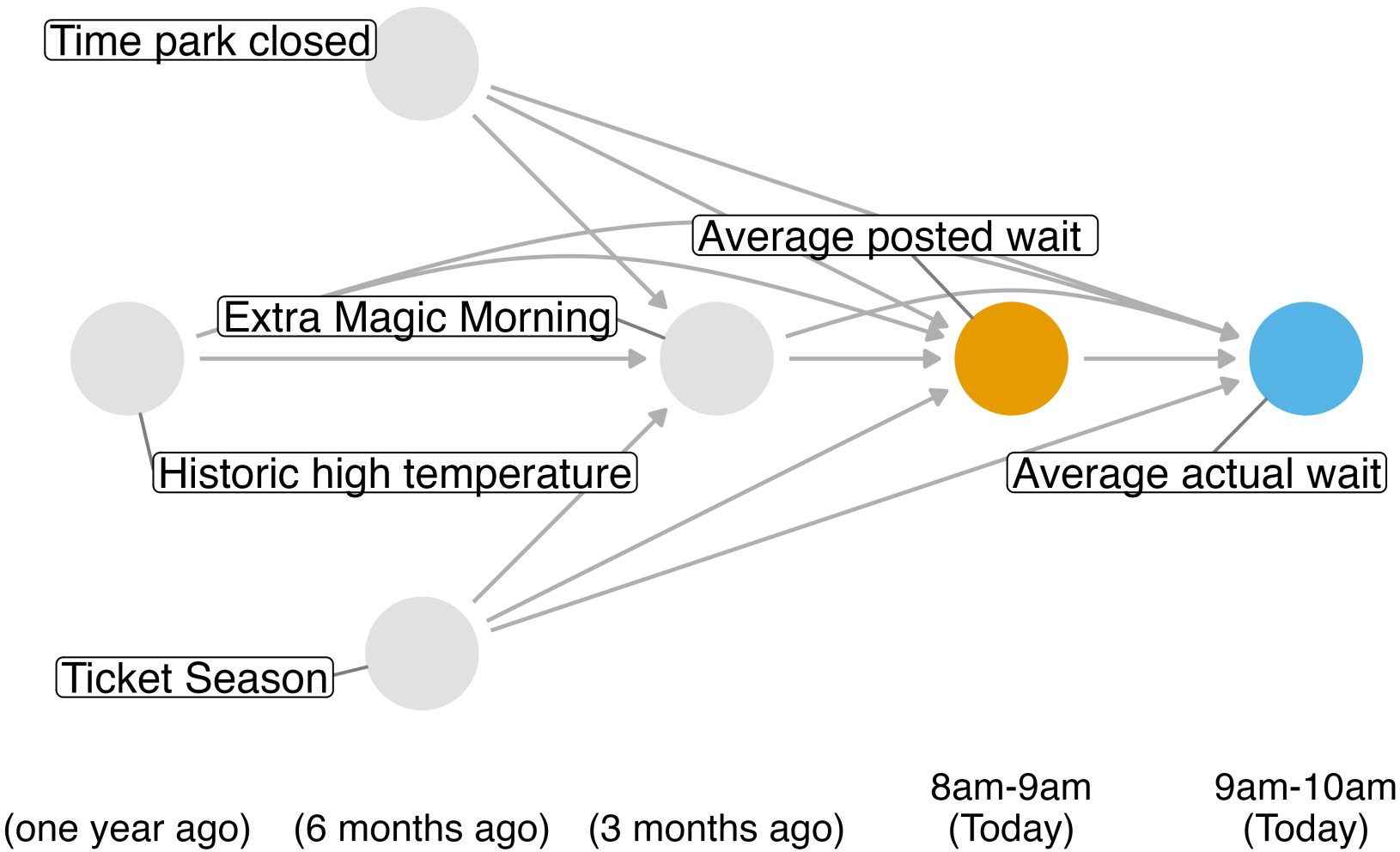
```
nhefs_wts <- nhefs_model |>
  augment(data = nhefs_light_smokers) |>
  mutate(wts = wt_ate(
    .fitted,
    smkintensity82_71,
    exposure_type = "continuous"
  ))
```

2. Calculate the weights with `wt_ate()`

```
1 nhefs_wts
```

```
# A tibble: 1,162 × 74
    seqn  qsmk death yrdth modth dadth   sbp   dbp sex
   <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <fct>
 1   235     0     0    NA    NA    NA   123    80 0
 2   244     0     0    NA    NA    NA   115    75 1
 3   245     0     1    85     2    14   148    78 0
 4   252     0     0    NA    NA    NA   118    77 0
 5   257     0     0    NA    NA    NA   141    83 1
 6   262     0     0    NA    NA    NA   132    69 1
 7   266     0     0    NA    NA    NA   100    53 1
 8   419     0     1    84    10    13   163    79 0
 9   420     0     1    86    10    17   184   106 0
10   434     0     0    NA    NA    NA   127    80 1
# ℹ 1,152 more rows
```

Do *posted* wait times at 8 am affect *actual* wait times at 9 am?



Your Turn 1

Fit a model using `lm()` with `wait_minutes_posted_avg` as the outcome and the confounders identified in the DAG.

Use `augment()` to add model predictions to the data frame

In `wt_ate()`, calculate the weights using `.fitted` and `wait_minutes_posted_avg`

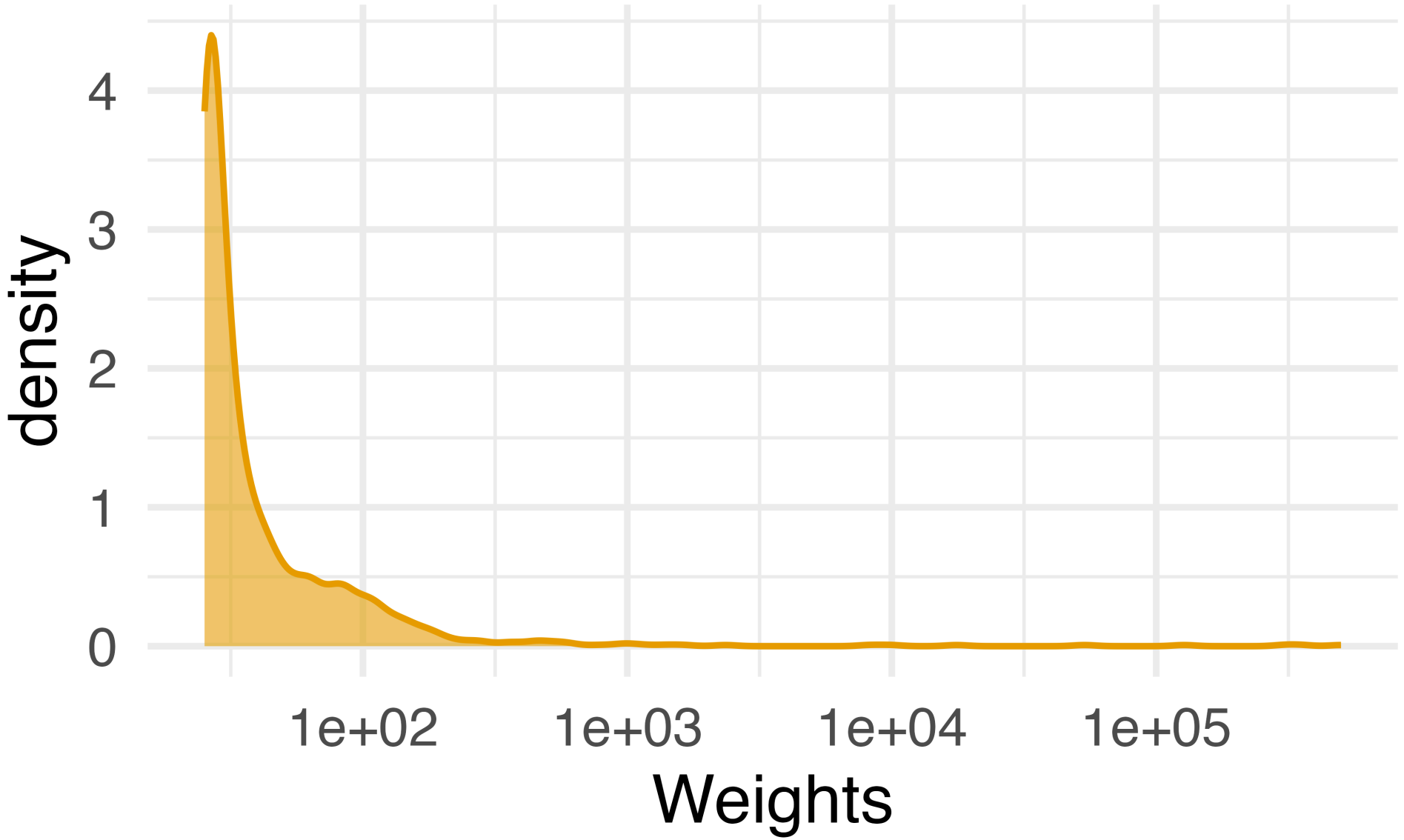
Your Turn 1

```
post_time_model <- lm(
  wait_minutes_posted_avg ~
    park_close +
    park_extra_magic_morning +
    park_temperature_high +
    park_ticket_season,
  data = wait_times
)
```

Your Turn 1

```
wait_times_wts <- post_time_model |>
  augment(data = wait_times) |>
  mutate(
    wts = wt_ate(
      .fitted,
      wait_minutes_posted_avg,
      exposure_type = "continuous"
    )
  )
```

Stabilizing extreme weights



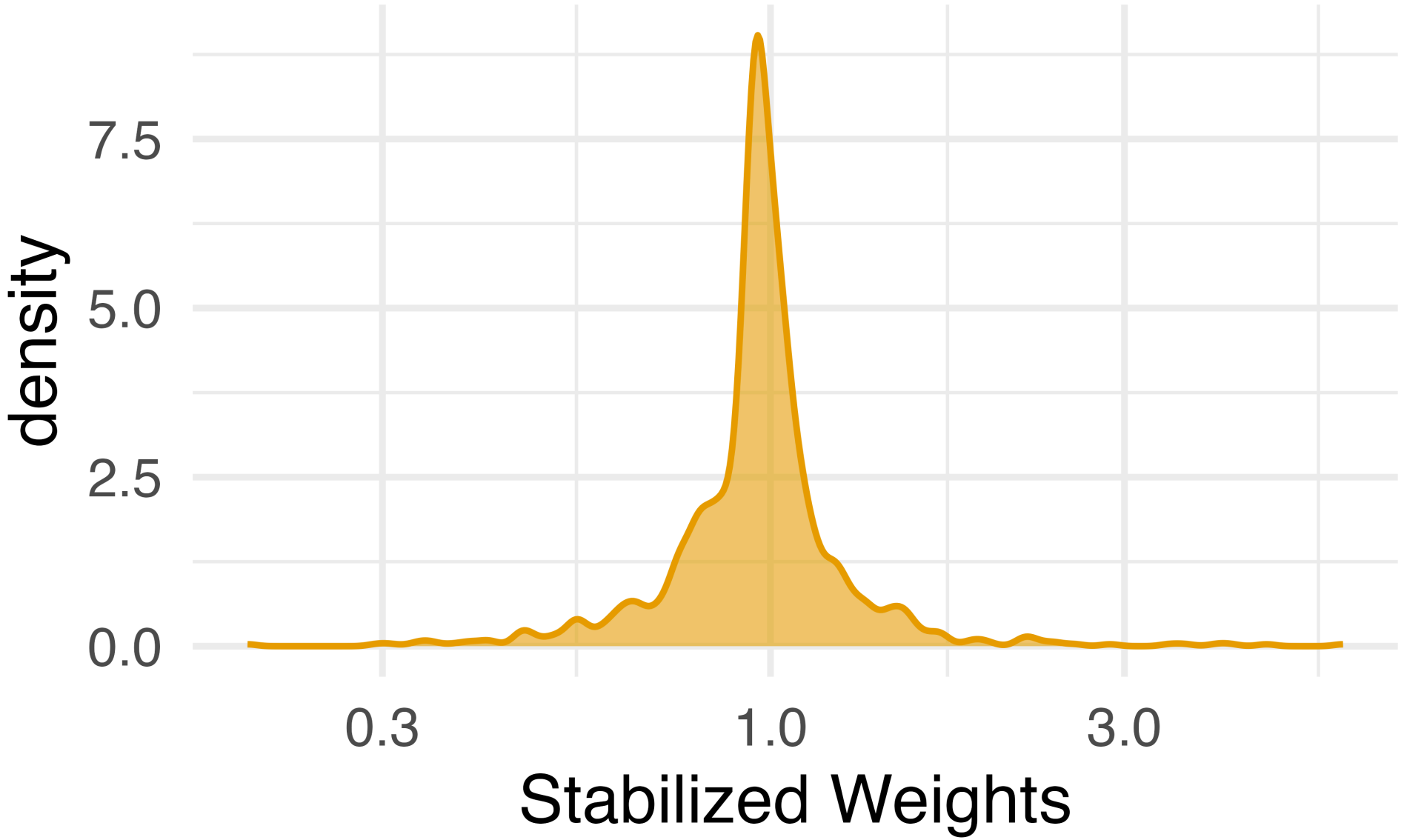
Stabilizing extreme weights

1. Fit an intercept-only model (e.g. `lm(x ~ 1)`) or use mean and SD of `x`
2. Calculate the marginal density from this model.
3. Divide the marginal density by the propensity score density. `wt_ate(..., stabilize = TRUE)` does this all!

Calculate stabilized weights

```
nhefs_swts <- nhefs_model |>
  augment(data = nhefs_light_smokers) |>
  mutate(swts = wt_ate(
    .fitted,
    smkintensity82_71,
    exposure_type = "continuous",
    stabilize = TRUE
  ))
```

Stabilizing extreme weights



Stabilized weights should have a mean close to 1

```
1 mean(nhefs_swts$swts)
```

```
[1] 0.9989975
```

If the mean is far from 1, we may have issues with model misspecification or positivity violations. A check worth running every time.

Your Turn 2

Re-fit the above using stabilized weights

Your Turn 2

```
wait_times_swts <- post_time_model |>
  augment(data = wait_times) |>
  mutate(
    swts = wt_ate(
      .fitted,
      wait_minutes_posted_avg,
      exposure_type = "continuous",
      stabilize = TRUE
    )
  )
```

Checking balance

1. Weighting should break the association between the confounders and the exposure
2. For a binary exposure, we check standardized mean differences
3. For a continuous exposure, we check correlations and energy

Check balance with `check_balance()`

```
balance_metrics <- check_balance(
  nhefs_swts,
  .vars = c(age, smokeintensity, smokeyrs, wt71),
  .exposure = smkintensity82_71,
  .weights = swts,
  exposure_type = "continuous"
)
```

These are the continuous confounders; the categorical ones need dummy coding first

As in `wt_ate()`, `exposure_type` tells `check_balance()` the exposure is continuous

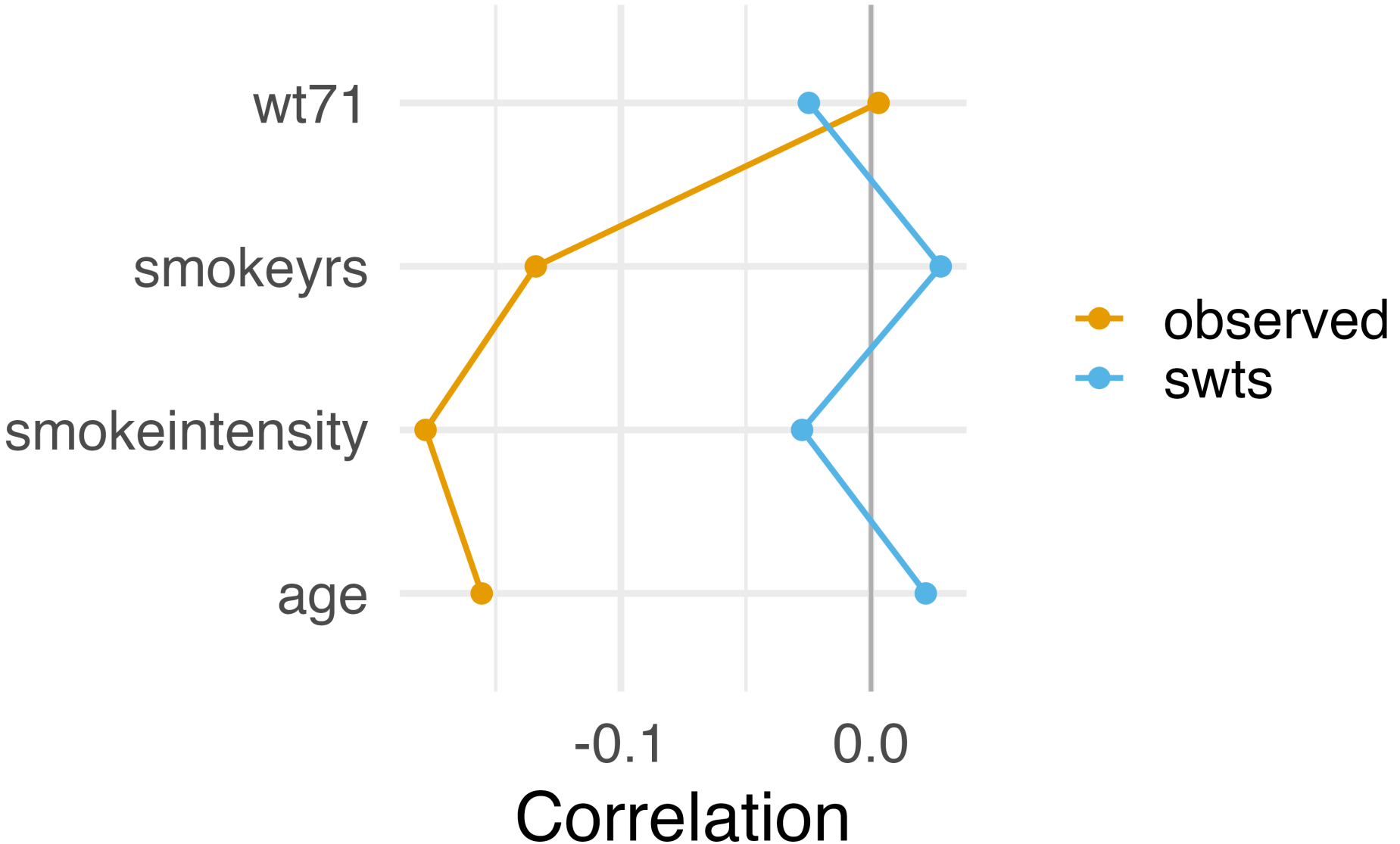
Check balance with `check_balance()`

```
1 balance_metrics
```

```
# A tibble: 10 × 5
   variable        group_level         method   metric estimate
   <chr>           <chr>               <chr>    <chr>     <dbl>
 1 age             smkintensity82_71   observed corre… -0.156
 2 age             smkintensity82_71   swts     corre…  0.0219
 3 smokeintensity  smkintensity82_71   observed corre… -0.178
 4 smokeintensity  smkintensity82_71   swts     corre… -0.0276
 5 smokeyrs        smkintensity82_71   observed corre… -0.134
 6 smokeyrs        smkintensity82_71   swts     corre…  0.0279
 7 wt71            smkintensity82_71   observed corre…  0.00301
 8 wt71            smkintensity82_71   swts     corre… -0.0248
 9 <NA>            <NA>                observed energy   2.03
10 <NA>            <NA>                swts     energy   0.798
```

The weighted correlations are close to zero, and energy is smaller after weighting

Correlations before and after weighting



Fitting the outcome model

1. Use the stabilized weights in the outcome model. Nothing new here!
2. Use `ipw()` for standard errors that account for the weights

Fit the outcome model

```
nhefs_ipw_model <- lm(
  wt82_71 ~ smkintensity82_71,
  weights = swts,
  data = nhefs_swts
)
```

```r
nhefs_ipw_model |>
  tidy() |>
  filter(term == "smkintensity82_71") |>
  mutate(estimate = estimate * -10)
```

```
# A tibble: 1 × 5
  term              estimate std.error statistic    p.value
  <chr>                <dbl>     <dbl>     <dbl>      <dbl>
1 smkintensity82_71    0.965    0.0206     -4.68 0.00000323
```

Estimate the effect with `ipw()`

```
1 nhefs_ipw_fit <- ipw(nhefs_model, nhefs_ipw_model)
```

`ipw()` uses a sandwich estimator that also accounts for the uncertainty in the propensity score model

Estimate the effect with `ipw()`

```
1 nhefs_ipw_fit
```

```
Inverse Probability Weight Estimator
Estimand: ATE
Effects: marginal (population-averaged)

Weight Estimator:
  Call: lm(formula = smkintensity82_71 ~ sex + race + age + I(age^2) +
    education + smokeintensity + I(smokeintensity^2) + smokeyrs +
    I(smokeyrs^2) + exercise + active + wt71 + I(wt71^2), data = nhefs_light_smokers)

Outcome Model:
  Call: lm(formula = wt82_71 ~ smkintensity82_71, data = nhefs_swts,
    weights = swts)

Marginal estimates:
       estimate   std.err       z ci.lower  ci.upper conf.level  p.value
slope -0.096497  0.036518 -2.6424 -0.16807 -0.024923       0.95 0.008231 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Your Turn 3

Estimate the relationship between posted wait times and actual wait times using the stabilized weights we just created, saving the model to an object

Use `ipw()` to estimate the effect with standard errors that account for the weights, then `as_conditional()` to switch readings

Your Turn 3

```r
ipw_model <- lm(
  wait_minutes_actual_avg ~ wait_minutes_posted_avg,
  weights = swts,
  data = wait_times_swts
)

ipw_model |>
  tidy() |>
  filter(term == "wait_minutes_posted_avg") |>
  mutate(estimate = estimate * 10)
```

```
# A tibble: 1 × 5
  term                  estimate std.error statistic p.value
  <chr>                    <dbl>     <dbl>     <dbl>   <dbl>
1 wait_minutes_posted_…    -2.91    0.0725     -4.01 1.30e-4
```

Your Turn 3

```
1 ipw_fit <- ipw(post_time_model, ipw_model)
```

`ipw()` reports the effect of one more minute, so multiply by 10 to compare

Your Turn 3

```
1 ipw_fit
```

```
Inverse Probability Weight Estimator
Estimand: ATE
Effects: marginal (population-averaged)

Weight Estimator:
  Call: lm(formula = wait_minutes_posted_avg ~ park_close + park_extra_magic_morning +
    park_temperature_high + park_ticket_season, data = wait_times)

Outcome Model:
  Call: lm(formula = wait_minutes_actual_avg ~ wait_minutes_posted_avg,
    data = wait_times_swts, weights = swts)

Marginal estimates:
      estimate  std.err      z ci.lower  ci.upper conf.level  p.value
slope -0.29116  0.10991 -2.649 -0.50659 -0.075731       0.95 0.008074 **
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Your Turn 3

```
1 as_conditional(ipw_fit) |>
2   tidy(conf.int = TRUE)
```

```
# A tibble: 2 × 7
  term         estimate std.error statistic  p.value conf.low
  <chr>           <dbl>     <dbl>     <dbl>    <dbl>    <dbl>
1 (Intercept)   44.9        4.51       9.95 2.41e-23   36.0
2 wait_minut…   -0.291      0.110     -2.65 8.07e- 3   -0.507
# ℹ 1 more variable: conf.high <dbl>
```

The slope matches the marginal reading here because the outcome model is linear in the exposure with an identity link

Causal dose-response functions

1. Calculate the stabilized weights
2. Fit the outcome model with a flexible term for the exposure
3. Use `ipw()` for standard errors that account for the weights
4. Predict the outcome across the range of the exposure

1. Calculate the stabilized weights

`nhefs_swts` already has the stabilized weights

2. Fit a flexible outcome model

```r
nhefs_msm <- lm(
  wt82_71 ~ splines::ns(smkintensity82_71, df = 3),
  weights = swts,
  data = nhefs_swts
)
```

3. Estimate the effects with `ipw()`

```
nhefs_ipw <- ipw(
  nhefs_model,
  nhefs_msm,
  .data = nhefs_swts
)
```

The standard errors account for the estimated weights.

4. Predict across the exposure range

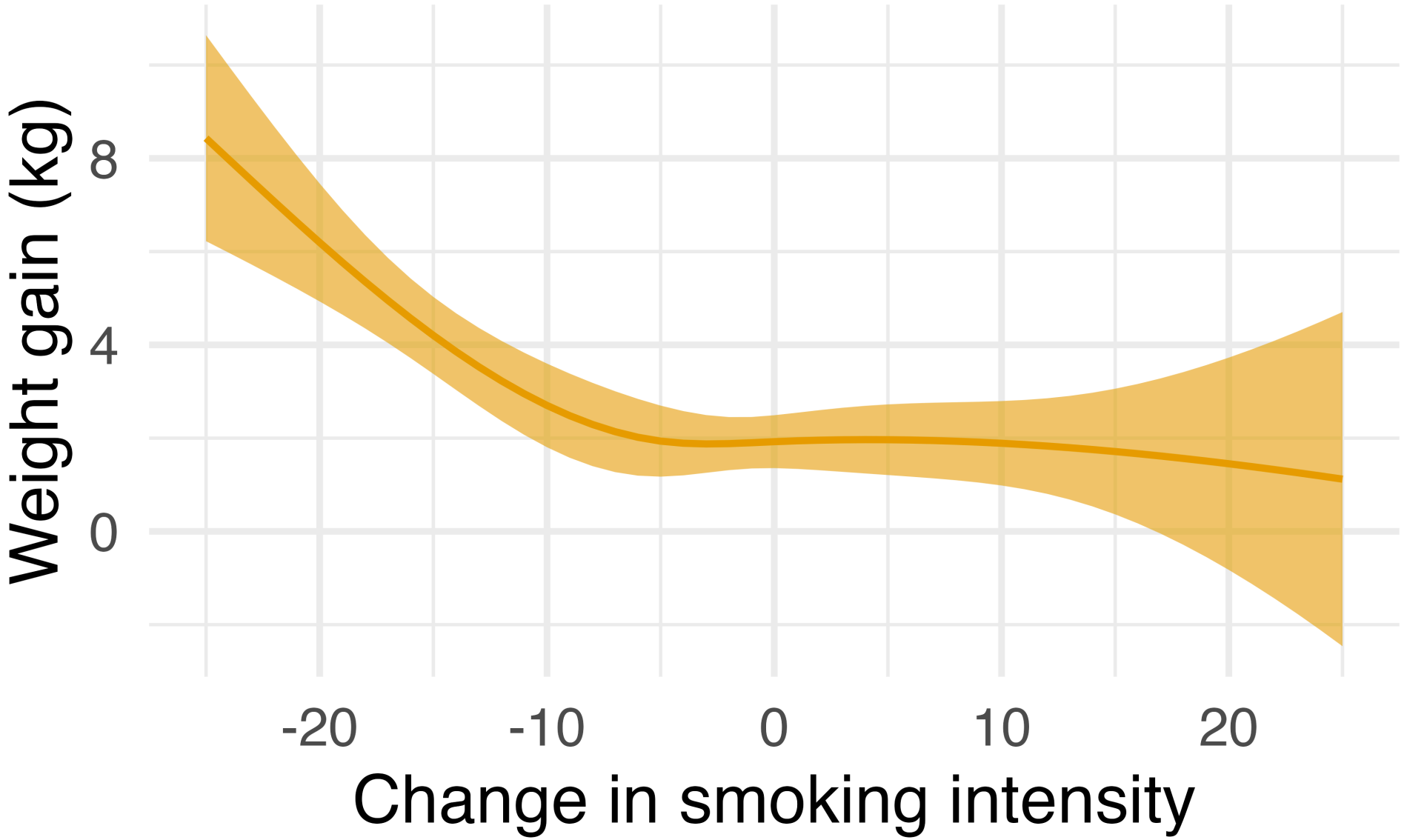
```
dose_response <- nhefs_ipw |>
  as_conditional() |>
  # marginaleffects needs the data to build the grid
  set_modeldata(nhefs_swts) |>
  avg_predictions(
    variables = list(smkintensity82_71 = -25:25)
  )

dose_response
```

The predictions along the exposure grid come from the outcome model itself, so we switch to the conditional reading first

4. Predict across the exposure range

```
 smkintensity82_71 Estimate Std. Error     z Pr(>|z|)    S 2.5 % 97.5 %
               -25     8.43      1.126 7.485   <0.001 43.7  6.22  10.64
               -24     7.98      1.021 7.815   <0.001 47.4  5.98   9.98
               -23     7.52      0.918 8.194   <0.001 51.8  5.72   9.32
               -22     7.07      0.820 8.626   <0.001 57.1  5.47   8.68
               -21     6.63      0.728 9.103   <0.001 63.3  5.20   8.06
--- 41 rows omitted. See ?print.marginaleffects ---
                21     1.39      1.282 1.084    0.278  1.8 -1.12   3.90
                22     1.33      1.409 0.941    0.347  1.5 -1.44   4.09
                23     1.26      1.542 0.817    0.414  1.3 -1.76   4.28
                24     1.19      1.682 0.708    0.479  1.1 -2.11   4.49
                25     1.12      1.828 0.613    0.540  0.9 -2.46   4.70
Type: response
```

The causal dose-response function



Diagnosing issues

1. Extreme weights even after stabilization
2. Bootstrap: non-normal distribution
3. Bootstrap: estimate different from original model

More info

Causal Inference in R: **g-computation for continuous exposures**

https://github.com/LucyMcGowan/writing-positivity-continous-ps